Dispositivo Vestível para Pessoas com Necessidades Especiais: SoC RISC-V com Coprocessador de Fusão Sensorial de Oito Canais, da Verificação UVM e Síntese com DFT ao Protótipo em FPGA

Kyara Esteves de Sousa
Programa CI-Digital – Residência em TIC 43
Brasília, DF, Brasil

***Resumo*—Este trabalho apresenta o núcleo eletrônico de um dispositivo vestível para pessoas com necessidades especiais, em particular pessoas com deficiência visual: um sistema-em-chip (SoC) que integra o processador RISC-V FemtoRV32 (RV32I), 16 KB de memória em macros de SRAM, sensores de distância ultrassônico (HC-SR04) e laser de tempo de voo (VL53L0X) e um coprocessador que implementa em hardware a fusão por filtro de Kalman estático. Os blocos se comunicam por AXI4-Lite. O núcleo de fusão foi projetado com oito nós que operam simultaneamente, e o mesmo RTL foi verificado por testes dirigidos, simulação de sistema e cinco ambientes UVM (0 erros, cobertura funcional de 90,1% a 99,4%). Ele foi sintetizado no Cadence Genus (GPDK045) com e sem cadeias de varredura e prototipado em uma FPGA Gowin GW2A-18C, que fecha timing a 74,8 MHz usando 11% da lógica.**

A 27 MHz, um nó calcula ganho, estimativa e variância em 22 ciclos, 102 vezes menos que a rotina equivalente em software. Visto pelo programa, incluindo o barramento, o coprocessador é 12,5 vezes mais rápido e gasta 12,5 vezes menos energia por fusão, com resultado bit a bit idêntico ao software. Com os oito nós em paralelo a 200 MHz, o núcleo atinge 72,7 milhões de fusões por segundo, cerca de 20 vezes a vazão da arquitetura de referência. Uma calibração em seis distâncias reduziu a divergência entre os sensores de cerca de 50 mm para ±5 mm. A DFT torna 100% dos 2.325 registradores varríveis, com acréscimo de 5,9% na área total.

Foram executadas e documentadas, com relatórios e *logs* das ferramentas:

- **testes funcionais pré-síntese (RTL) no Xcelium;**
- **síntese lógica no Cadence Genus, com e sem DFT;**
- **simulação pós-síntese da *netlist* com SDF (memória);**
- **UVM dos cinco blocos do núcleo de fusão;**
- **inserção de DFT no SoC inteiro, incluindo o núcleo de fusão, que contribui com 570 flip-flops na cadeia de varredura.**

***Index Terms*—dispositivo vestível, tecnologia assistiva, RISC-V, AXI4-Lite, fusão sensorial, filtro de Kalman, coprocessador, UVM, DFT, síntese lógica, FPGA, calibração.**

I. Introdução

Dispositivos vestíveis assistivos para pessoas com necessidades especiais, como as com deficiência visual, estimam a distância até obstáculos com sensores baratos, cada um sujeito a falhas diferentes. O ultrassom é pouco afetado por vidro e superfícies escuras, mas sofre com a geometria e a temperatura do alvo. O laser de tempo de voo é rápido e preciso, mas falha com superfícies transparentes. Fundir as duas medidas produz uma estimativa melhor e com incerteza conhecida [1], [3].

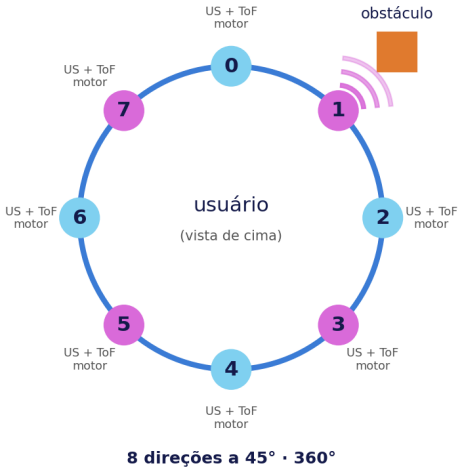


Figura 1. Aplicação: aro tátil com oito módulos (ultrassom + laser ToF e motor de vibração) a cada 45°. A direção 1, com um obstáculo próximo, vibra com mais intensidade.

A aplicação-alvo é um aro tátil vestível (Fig. 1):

- oito módulos de sensores, cada um com um ultrassom e um laser de tempo de voo, distribuídos a cada 45° ao redor do corpo, cobrindo 360°;
- cada direção i é fundida por um nó de fusão, produzindo a distância estimada $\hat{q}[i]$;
- o processador lê $\hat{q}[i]$ e define a intensidade de um motor de vibração (ERM ou LRA) na mesma posição: quanto mais próximo o obstáculo, mais forte a vibração naquela direção, uma realimentação tátil intuitiva e imediata.

Este trabalho implementa e valida o núcleo eletrônico desse aro. O núcleo de oito nós foi verificado e sintetizado, e uma direção completa (dois sensores reais e um nó) roda em FPGA. O acionamento dos motores fica como etapa seguinte.

Conde *et al.* [1] implementaram a fusão estática por filtro

de Kalman em FPGA, para um par de sensores. O módulo de fusão produzia um resultado a cada 14 ciclos a 50 MHz (0,28 µs), cerca de 514 vezes mais rápido que a mesma conta num processador NIOS II.

Este trabalho complementa essa referência em quatro frentes:

- **Escala:** o núcleo de fusão passa de um para **oito canais simultâneos** (Seção III-D).
- **Integração:** o núcleo vira um coprocessador mapeado em memória de um SoC RISC-V com barramento AXI4-Lite, com mapa de registros e rotina de software (Seção III).
- **Fluxo de CI digital completo:** verificação em camadas e UVM com métricas de latência e vazão (Seções IV e V), e síntese lógica no Cadence Genus com e sem DFT (Seções VI e VII).
- **Protótipo:** o SoC roda em FPGA com sensores reais, e um procedimento de calibração separa viés e variância (Seções VIII e VIII-A).

II. Fusão por Filtro de Kalman Estático

Sejam q_1 e q_2 duas medidas da mesma distância, com variâncias σ_1^2 e σ_2^2. A estimativa de mínimos quadrados ponderados, escrita na forma do filtro de Kalman [1], [2], é

$$K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}, \tag{1}$$

$$\hat{q} = q_1 + K\,(q_2 - q_1), \tag{2}$$

$$\sigma^2 = \sigma_1^2 - K\,\sigma_1^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2}. \tag{3}$$

A variância fundida σ^2 é sempre menor que as duas de entrada. A formulação supõe erros *de média zero*: um viés sistemático não é corrigido pelo filtro e precisa ser removido antes, por calibração (Seção VIII-A).

No hardware, K é representado em ponto fixo UQ1.15 (32.768 = 1,0); medidas e variâncias são inteiros de 16 bits, em mm e mm². Cada operação é arredondada uma única vez (meio para cima) e saturada.

III. Arquitetura do SoC

A Fig. 2 mostra o SoC completo. O RTL (Verilog-2001 e SystemVerilog) é o mesmo para o ASIC e para a FPGA; só a memória muda, escolhida por um parâmetro.

A. Processador

Fonte. O processador é o FemtoRV32 Quark, de Bruno Levy e Matthias Koch (2020–2021) [5]. Ele vem do repositório *learn-fpga*, arquivo `FemtoRV/RTL/PROCESSOR/femtorv32_quark.v`, distribuído sob a licença BSD de três cláusulas, que permite uso e modificação com preservação do aviso de *copyright*.

O Quark é a variante mais elementar da família FemtoRV32. Ele implementa a ISA RV32I [4] mais a leitura do contador de ciclos (`RDCYCLE`), num único arquivo Verilog de cerca de 400 linhas. É um núcleo multiciclo, sem *pipeline*, escrito para FPGA.

O arquivo original está preservado sem alteração em `frontend/upstream/`, junto com um *diff* das mudanças. Cada mudança no RTL usado está marcada no código como `[ASIC-1]` a `[ASIC-4]`. Para o fluxo ASIC, o núcleo recebeu quatro adaptações, sem mudar a ISA nem a temporização:

- declaração antes do uso, exigida por analisadores estritos do padrão;
- reset síncrono em `instr`, `aluShamt`, `aluReg` e no contador de ciclos, que em ASIC iniciam em X e travavam a máquina de estados na simulação pós-síntese;
- leitura de `x0` forçada a zero;
- renomeação do módulo.

B. Barramento AXI4-Lite

O AXI4-Lite é o perfil simplificado da especificação AMBA AXI4 [6]: uma palavra por transação, sem rajadas, sem IDs e sem transações fora de ordem. Ele mantém os cinco canais independentes (AW, W, B, AR, R), com handshake `VALID`/`READY`, e é compatível com interconexões AXI4 completas. O processador emite no máximo uma transação por vez, então rajadas não trariam ganho nesta versão.

Quatro blocos formam o barramento:

- **Ponte** (`femto_axil_master`): converte a interface nativa do Quark em mestre AXI4-Lite.
- **Crossbar** (`axil_xbar`): decodifica o endereço e responde `DECERR` fora do mapa.
- **Conversor** (`axil2reg`): adapta o AXI4-Lite a registradores simples nos periféricos.
- **Escravo de memória** (`axil_sram`): encapsula a RAM.

A Fig. 3 mostra uma rodada do firmware: cada acesso, a instrução, o sensor ou o coprocessador, é uma transação na *crossbar*.

C. Memória: origem e integração

O Genus não gera SRAM: um vetor em RTL vira flip-flops. A GPDK045 também não inclui compilador de memória. Por isso a memória foi gerada com o compilador de código aberto OpenRAM [7], [8], versão 1.2.48, desenvolvido pelos grupos VLSI-DA da UC Santa Cruz e da Oklahoma State University e distribuído sob a licença BSD de três cláusulas. A tecnologia-alvo foi o FreePDK45 [9], um PDK acadêmico de 45 nm da North Carolina State University. A biblioteca de células padrão do projeto é a `gsclib045` do kit genérico GPDK045 da Cadence [10], que não inclui compilador de memória.

O compilador foi instalado pelo `pip` e executado com o arquivo de configuração guardado em `tech/sram/.../doc/openram_config.py`, com estes parâmetros:

- 1.024 palavras de 32 bits;
- máscara de escrita por byte;
- uma porta de leitura e escrita;
- modelo de atraso analítico;
- corner único TT (1,0 V, 25 °C);
- *layout* completo, com DRC e LVS não executados.

O compilador entregou:

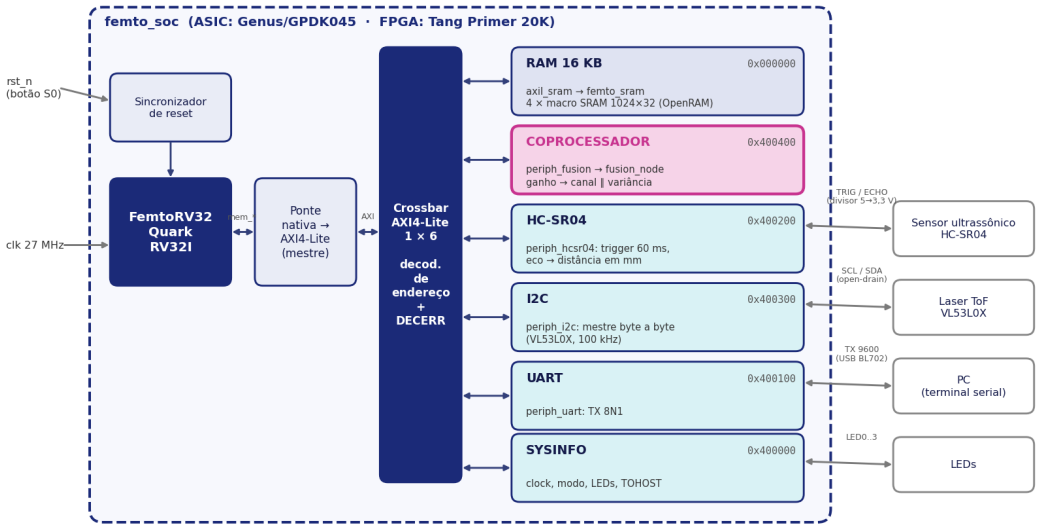


Figura 2. Arquitetura do SoC: o FemtoRV32 acessa todos os blocos pela ponte e pela *crossbar* AXI4-Lite. A RAM é formada por macros de SRAM no ASIC e por BSRAM na FPGA; os periféricos ligam o SoC aos sensores e ao computador.

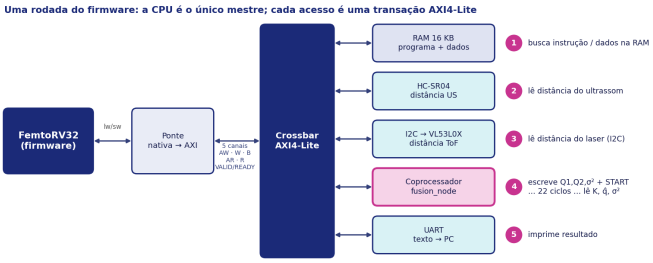


Figura 3. Comunicação no SoC: passos de uma rodada do firmware, todos como transações AXI4-Lite iniciadas pela CPU.

- o *layout* (GDS);
- a vista física (LEF, $237{,}9 \times 213{,}4\,\mu\text{m}$);
- o modelo de *timing* e área (Liberty, $50.766\,\mu\text{m}^2$);
- um modelo Verilog;
- um *datasheet*.

A integração no fluxo Cadence exigiu três ajustes:

- as camadas do LEF (`metalN`) foram renomeadas para a nomenclatura da GPDK045 (`MetalN`);
- a célula, marcada como `dont_use` no Liberty, foi liberada no `tech.tcl`;
- o atraso do modelo Verilog, marcado no próprio arquivo como arbitrário (3 ns), foi trocado pelos 0,4 ns do Liberty.

No Genus, a macro entra como caixa-preta. O *log* confirma a célula liberada e as instâncias resolvidas, e o relatório de referências não resolvidas fica vazio (Fig. 4).

A macro registra o pedido na borda de subida e entrega o dado na de descida, reproduzindo a latência de um ciclo que o Quark espera. São quatro macros, 16 KB no total.

Como limitação, o modelo é do FreePDK45 e analítico, não do silício da GPDK045. Isso explica as violações de capacitância máxima nos pinos da macro (Seção VI).

D. Núcleo de fusão: oito nós de três blocos

O `fusion_core` (Fig. 5) instancia, por um `generate`, oito `fusion_node` idênticos, um por par de sensores, que recebem dados e produzem resultados ao mesmo tempo. Cada nó tem três blocos:

- `fusion_gain`: calcula K (eq. 1) com um divisor restaurador sequencial de 17 ciclos;
- `fusion_channel`: calcula $\hat{q}$ (eq. 2) em 3 ciclos;
- `fusion_variance`: calcula σ^2 (eq. 3) em 2 ciclos.

Quando o ganho termina (`gain_valid`), canal e variância rodam em paralelo com o mesmo K. Um nó produz um resultado completo em 21 ciclos, ou 22 contando o registro do START.

No SoC desta etapa, o coprocessador `periph_fusion` usa um nó, com a mesma interface dos oito.

E. Mapa de registros e software

A Tabela I traz o mapa de registros. O firmware, em C para RV32I e com 9 KB, executa:

1) um autoteste que compara hardware e software em 64 vetores;
2) a inicialização do VL53L0X por I2C, com um porte da biblioteca da Pololu [14];
3) em laço: leitura dos sensores e correção pela calibração;
4) a fusão em software (`fusion_sw`, bit a bit equivalente ao hardware) e depois no coprocessador (`fusion_hw`: escreve as quatro entradas, dispara o START, espera o *done* e lê três resultados);
5) a medição dos dois caminhos com `rdcycle` e a impressão pela UART.

```
tech/sram/sram_32x1024_freepdk45/  (gerada pelo compilador OpenRAM)
 # OpenRAM 1.2.48 -- doc/openram_config.py
 word_size = 32
 num_words = 1024
 write_size = 8        # mascara de byte
 num_rw_ports = 1
 tech_name = "freepdk45"
 analytical_delay = True

 # sram_32x1024_freepdk45.html (datasheet gerado)
 WORD_SIZE 32 | NUM_WORDS 1024 | NUM_RW_PORTS 1
 Area (um2) 50766 | VDD 1.0 V | 25 C (TT)

work/femto_sram/genus.log + reports/femto_sram/  (Cadence Genus)
 SRAM macro liberada: lib_cell:ls_of_ld_worst/sram_32x1024_freepdk45_TT_1p0V_25C_lib/
     sram_32x1024_freepdk45  area=50766.39871875  avoid=false
 Instancias da macro sram_32x1024_freepdk45: 2 -> {g_bank[0].u_sram} {g_bank[1].u_sram}

 femto_sram_gates.rpt:
 sram_32x1024_freepdk45   2  101532.797  sram_32x1024_freepdk45_TT_1p0V_25C_lib  worst

 femto_sram_unresolved.rpt:
 No unresolved references in design 'femto_sram'
```

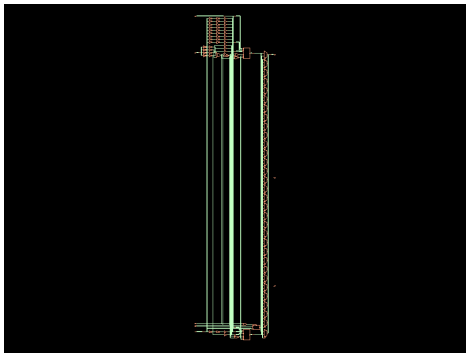


Figura 4. Origem e integração da memória: configuração do OpenRAM, *datasheet* gerado, *log* e relatórios do Genus da `femto_sram` (esq.; trechos copiados dos arquivos) e esquemático da `femto_sram` com as duas macros, gerado pelo Genus (dir.).

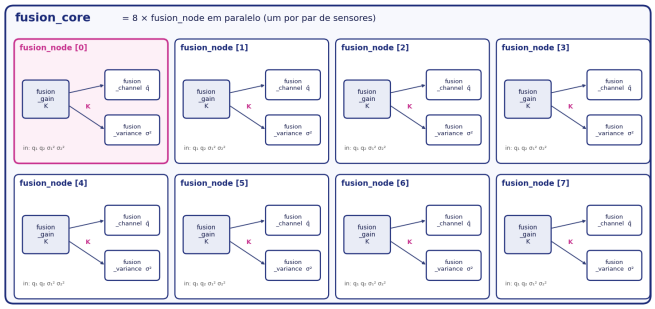


Figura 5. Núcleo de fusão: o `fusion_core` replica oito `fusion_node` que operam simultaneamente. Cada nó é composto por `fusion_gain` (eq. 1), `fusion_channel` (eq. 2) e `fusion_variance` (eq. 3).

Tabela I
MAPA DE REGISTROS DO COPROCESSADOR (BASE `0x0040_0400`)

Offset	Nome	Acesso	Descrição
0x00	Q1	RW	medida 1 (ultrassom), mm
0x04	Q2	RW	medida 2 (laser ToF), mm
0x08	SIGMA1_SQ	RW	variância do sensor 1, mm²
0x0C	SIGMA2_SQ	RW	variância do sensor 2, mm²
0x10	CTRL	W	bit 0: START
0x14	STATUS	R	bit 0: busy; bit 1: done
0x18	K	R	ganho, UQ1.15
0x1C	QHAT	R	estimativa fundida, mm
0x20	SIGMA_NEW	R	variância fundida, mm²
0x24	LAT	R	ciclos do START ao resultado
0x28	ID	R	`0xF0510001`

Mapa do SoC: RAM em `0x000000` (16 KB), sysinfo em `0x400000`, UART em `0x400100`, HC-SR04 em `0x400200`, I2C em `0x400300` e fusão em `0x400400`.

Em software, a eq. (1) exige uma divisão de 32 bits e as eqs. (2)–(3), duas multiplicações, todas emuladas pela `libgcc` no RV32I. Isso dá cerca de 2.200 ciclos por fusão e justifica a aceleração em hardware.

IV. VERIFICAÇÃO FUNCIONAL

A. *Testes dirigidos*

O processador foi verificado com dois programas:

- `isa_test`: cerca de 730 casos autoverificáveis de todas as instruções RV32I, com valores esperados de um modelo de referência em Python;
- `c_suite`: dez programas em C, cujos resultados esperados vêm do mesmo código compilado no computador.

Os dois rodam com memória ideal e com estados de espera aleatórios. Com estados de espera:

- o `isa_test` executou 5.476 instruções em 28.969 ciclos (CPI 5,29);
- o `c_suite` executou 172.567 instruções (CPI 5,25), com os dez programas conferidos.

Num teste de mutação, dez defeitos injetados no RTL foram todos detectados.

A memória passou num testbench de quatro fases com 20 mil acessos aleatórios, em RTL e na netlist do Genus:

- macros: 12.157 leituras e 11.221 escritas conferidas, com todas as máscaras de byte;
- a mesma RAM em flip-flops: 10.445 leituras conferidas;
- o Quark executando o `c_suite` a partir das macros: 613.686 ciclos, PASS.

O SoC executa o firmware real com modelos dos sensores (HC-SR04 por largura de pulso; VL53L0X como escravo I2C):

- o autoteste compara 64 vetores, todos iguais;
- o VL53L0X é inicializado pelo I2C;
- quatro fusões com leituras modeladas dão 0 divergências entre hardware e software (2.239 contra 247 ciclos na versão inicial do firmware);
- 1.369 caracteres saem na UART, com 13 ecos do HC-SR04 e 45 leituras I2C, e a simulação termina com `RESULT: PASS`.

A regressão no Xcelium tem 8 cenários, todos aprovados. A configuração da FPGA (RAM em BSRAM) também foi simulada.

B. *Simulação pós-síntese*

Na simulação pós-síntese, o Xcelium executa a *netlist* gerada pelo Genus, com as células da GPDK045 e as macros, anotada com os atrasos do arquivo SDF escrito pela própria síntese.

```
MODULE=femto_sram ./exe.sh post   (Xcelium: netlist do Genus + SDF)
  Reading SDF file from location ".../deliverables/femto_sram/femto_sram_worsts.sdf"
  Annotating SDF timing data:
      Backannotation scope:  femto_sram_tb.dut
      MTM control:           MAXIMUM
  Annotation completed with 0 Errors and 256 Warnings
      Path Delays    186  disabled 0  annotated 186   100.00%
      $setuphold       4  disabled 0  annotated   4   100.00%
  [tb] DUT: femto_sram (macros OpenRAM), 2 bancos x 1024 palavras
  [tb] modo: POST-SYNTHESIS (netlist)
  [tb] fase 1 (preenche/le 2048 palavras): 0 erro(s)
  [tb] fase 2 (mascaras de byte/meia-palavra): 0 erro(s)
  [tb] fase 3 (sel=0 ignora escritas): 0 erro(s)
  [tb] fase 4 (aleatorio): 0 erro(s)
  [tb] leituras conferidas: 12317   escritas: 11172
  RESULT: PASS
  FUNCTIONAL_RESULT: PASS   [femto_sram mem post sdf=yes]

./run_all_functional.sh   (Xcelium, regressão RTL)
  RESUMO / pre (8/8 PASS, 0 pulados)
  PASS  femtorv32_quark  isa_test  ws=0
  PASS  femtorv32_quark  isa_test  ws=1
  PASS  femtorv32_quark  c_suite  ws=0
  PASS  femtorv32_quark  c_suite  ws=1
  PASS  femto_sram
  PASS  femto_ram_ff
  PASS  femtorv32_sram  c_suite_8k  ws=0
  PASS  femto_soc  soc_fw
  TUDO PASSOU.
```

Figura 6. *Log* do Xcelium (trechos): pós-síntese da `femto_sram` com SDF (anotação de 100% e PASS) e resumo da regressão RTL (8/8).

Tabela II
RESULTADOS DA UVM (0 `UVM_ERROR`/`UVM_FATAL` EM TODOS OS TESTES)

Ambiente	Testes	Transações	Cobertura funcional
`fusion_gain`	4	4.149	90,10%
`fusion_channel`	4	4.827	99,40%
`fusion_variance`	4	4.906	92,36%
`fusion_node`	5	6.096	94,44%
`fusion_core` (8 nós)	5	4.980	96,67%

Para a `femto_sram`, foram anotados 186 de 186 atrasos de caminho e 4 de 4 checagens de *setup*/*hold*. O mesmo testbench do RTL passou com 0 erros nas quatro fases, conferindo 12.317 leituras e 11.172 escritas (Fig. 6). Isso mostra que a síntese preservou o comportamento da memória com os atrasos reais das células.

A pós-síntese do SoC completo fica como trabalho futuro: os testbenches carregam o firmware por nomes hierárquicos que mudam na *netlist*.

C. UVM

Cada bloco do núcleo tem um ambiente UVM [11] com agente, *driver*, monitores, verificação de latência, cobertura funcional e um *scoreboard* com modelo de referência bit a bit. O *scoreboard* também confere as equações (1)–(3) em ponto flutuante, com erro máximo de 0,5 LSB.

Ao todo, os cinco ambientes executam 22 testes (*sanity*, *random*, *coverage*, *factory*, sequência virtual e arbitragem), todos aprovados, com mais de 24 mil transações conferidas. O ambiente do `fusion_core` confere os oito canais em toda transação, inclusive com valores diferentes por canal, para garantir que nada vaze entre eles. A Tabela II resume os resultados.

Tabela III
LATÊNCIA, VAZÃO E ENERGIA POR FUSÃO

Caminho	Ciclos	Tempo	Fusões/s
Software RV32I (27 MHz)	2.234	82,7 µs	12 mil
Coprocessador via AXI (27 MHz)	179	6,6 µs	151 mil
1 nó (27 MHz)	22	0,81 µs	1,23 M
8 nós em paralelo (200 MHz)	22	0,11 µs	**72,7 M**
Conde *et al.* [1], 1 canal (50 MHz)	14	0,28 µs	3,6 M

Blocos do nó: ganho 17, canal 3 e variância 2 ciclos. 8 nós: 8 fusões a cada 22 ciclos.

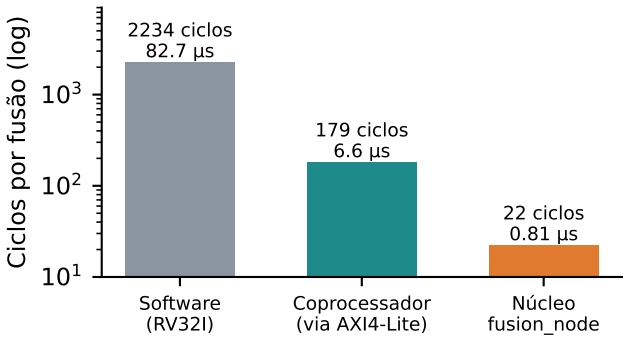


Figura 7. Ciclos por fusão medidos na FPGA: software no RV32I, coprocessador visto pelo programa e só o núcleo.

V. LATÊNCIA, VAZÃO E ENERGIA

A Tabela III reúne as métricas medidas na placa e em simulação. A latência do nó é dominada pelo divisor do ganho, que produz um bit do quociente por ciclo. As oito transações AXI4-Lite custam cerca de 157 ciclos com as instruções da CPU. Por isso a aceleração percebida pelo software (12,5×, Fig. 7) é menor que a do núcleo (102×).

A energia por fusão foi estimada como a potência vetorless do Genus multiplicada pelo tempo:

- SoC (1,387 mW): 115 nJ em software e 9,2 nJ com o coprocessador;
- núcleo (8,89 mW para os oito nós a 200 MHz): cerca de 0,12 nJ por fusão.

Comparação com a referência. Por fusão, a latência do nó (22 ciclos) é maior que a do módulo de Conde *et al.* (14 ciclos), porque a divisão é sequencial e em ponto fixo. Mas os oito nós operam ao mesmo tempo, e o `fusion_core` fecha a 200 MHz. Por isso a vazão chega a 72,7 milhões de fusões por segundo, cerca de 20 vezes a da referência, com oito pares de sensores atendidos simultaneamente.

VI. SÍNTESE LÓGICA

A síntese usou o Cadence Genus 22.16 com a GPDK045 (`gsclib045`, corner lento: 0,9 V e 125 °C), em modo físico (PLE), com as macros de SRAM como caixas-pretas. A Tabela IV resume cada parte e o sistema, e a Fig. 8 mostra os esquemáticos gerados pela ferramenta.

Núcleo de fusão. O `fusion_core` de oito nós fecha a 200 MHz, com caminho crítico de 4,8 ns do registrador de

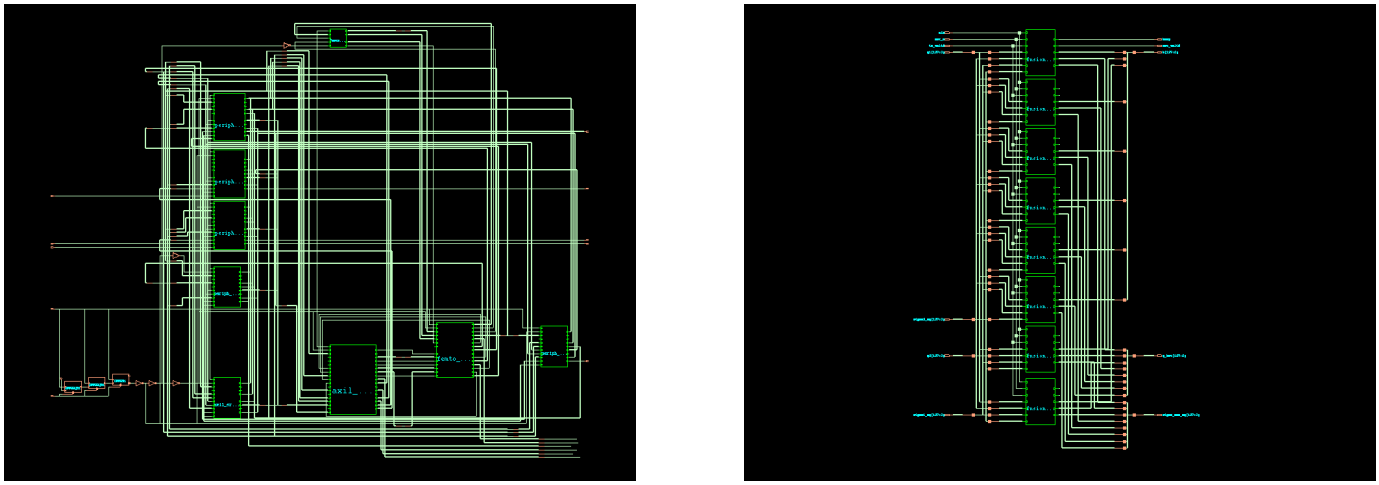


Figura 8. Esquemáticos gerados pelo Cadence Genus: (esq.) `femto_soc`, com a hierarquia preservada (periféricos, *crossbar*, RAM, CPU, ponte e sincronizador de reset); (dir.) `fusion_core` com os oito `fusion_node`.

Tabela IV
SÍNTESE LÓGICA NO GENUS (GPDK045, CORNER LENTO)

Design	Clock	Área (µm²)	Slack (ns)	Pot. (mW)
FemtoRV32 Quark	100 MHz	22.860	0,016	0,632
RAM 8 KB (2 macros)	100 MHz	101.936	3,908	1,965
RAM 1 KB (flip-flops)	100 MHz	108.557	6,665	1,894
`fusion_core` (8 nós)	200 MHz	92.025	0,000	8,887
SoC completo	27 MHz	**249.041**	**16,391**	**1,387**
SoC com DFT	27 MHz	**263.748**	**16,124**	**1,393**

Potência vetorless. Área total = células + fios.

ganho ao multiplicador da variância, e consome 8,89 mW. Dentro do SoC, a 27 MHz, um nó ocupa 9.464 µm² (43% da CPU), divididos entre ganho (1.524), canal (3.545) e variância (2.681). Com os registradores AXI, o coprocessador ocupa 12.727 µm².

Memória. A RAM de 1 KB em flip-flops ocupa mais área que 8 KB em macros: por KB, a macro é 8,5 vezes mais densa (Fig. 10).

SoC. O SoC fecha a 27 MHz com 16,4 ns de folga. O caminho crítico é a leitura da SRAM: o dado sai na borda de descida da macro e é capturado na subida pelo registrador da ponte AXI, um caminho de meio período com 1,87 ns de dados.

A área (Fig. 9) se divide em:

- memória: 82%;
- processador: 8,8%;
- coprocessador: 5,1%;
- demais periféricos e barramento: menos de 5%.

A potência se divide entre memória (82,8%), registradores (10,7%) e lógica (6,5%), com 9,7% de fuga. As únicas violações de regra de projeto são de capacitância máxima nas saídas das macros, um artefato do modelo analítico do OpenRAM.

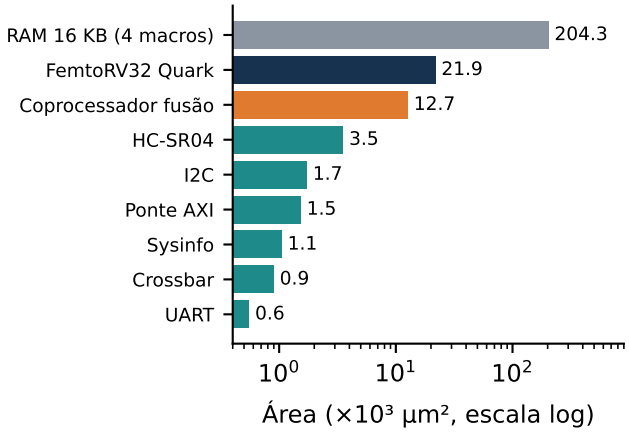


Figura 9. Área do SoC por bloco (síntese sem DFT).

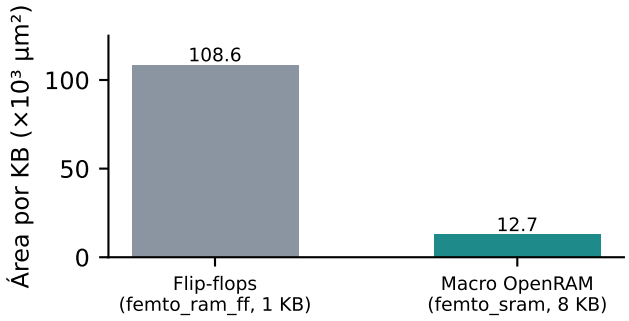


Figura 10. Área por KB: memória em flip-flops contra macros de SRAM.

VII. SÍNTESE COM DFT

A testabilidade foi inserida com varredura por multiplexador (*muxed scan*). O Genus verificou as regras de DFT antes e depois do mapeamento, conectou as cadeias, rodou uma

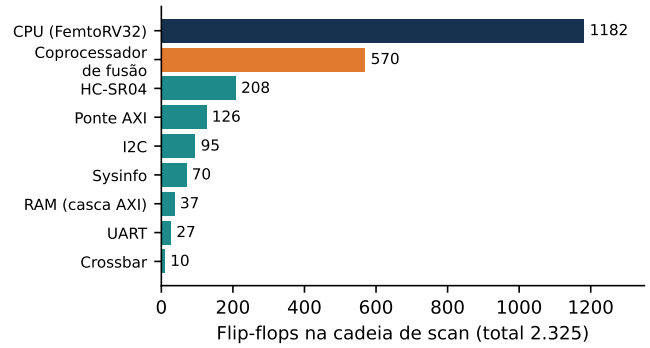


Figura 11. Flip-flops de cada bloco na cadeia de varredura do SoC com DFT (total 2.325). O coprocessador de fusão contribui com 570.

Tabela V
CUSTO DA DFT POR BLOCO (ÁREA TOTAL, µM²)

Bloco	Sem DFT	Com DFT	Δ
FemtoRV32 Quark	21.928	27.487	+25,4%
Coprocessador	12.727	16.811	+32,1%
`fusion_node`	9.464	12.001	+26,8%
Lógica total (sem macros)	44.753	59.198	+32,3%
SoC completo	249.041	263.748	+5,9%

otimização incremental e gerou os arquivos de ATPG para o Modus. O resultado:

- **0 violações de DFT**;
- **100 %** dos registradores varríveis;
- uma cadeia de **2.325** flip-flops, com as portas `SE`, `dft_sdi_1` e `dft_sdo_1`.

O único reset assíncrono vem de um pino primário e é controlável. As macros de SRAM ficam fora da cadeia: são caixas-pretas e, num projeto industrial, teriam BIST próprio.

A varredura cobre o SoC inteiro, inclusive o núcleo de fusão (Fig. 11). Contando os flip-flops por bloco no relatório da cadeia:

- processador: 1.182;
- coprocessador de fusão: 570, dos quais 358 no `fusion_node` (74 no ganho, 104 no canal e 50 na variância);
- periféricos e barramento: o restante.

Com os flip-flops de varredura, o nó passou de 9.464 para 12.001 µm² (+26,8%). A Fig. 12 traz os trechos do relatório e o esquemático da síntese com DFT.

Na lógica, a DFT custa 32% de área (Tabela V, Fig. 13). Como a memória domina o SoC, o impacto total é de 5,9%. A folga cai 0,27 ns e a potência sobe 0,4%.

VIII. PROTOTIPAGEM EM FPGA

O protótipo usa a Sipeed Tang Primer 20K [15] com placa Dock, baseada na FPGA Gowin GW2A-LV18PG256C8/I7, a 27 MHz. A família Gowin é suportada apenas pela ferramenta do fabricante (Gowin EDA), usada para síntese, posicionamento e roteamento e geração do *bitstream*. O RTL é o mesmo do ASIC e também pode ser sintetizado para Lattice.

Tabela VI
RECURSOS E DESEMPENHO NA GW2A-18C (GOWIN EDA)

Métrica	Valor
Lógica	2.211 (11%): 1.791 LUT + 420 ALU
Registradores	1.262 (8%)
CLS	1.546 (15%)
Blocos BSRAM	18 de 46 (40%)
DSP	1 de 24 (2 MULT18×18)
I/O	13 de 207
Fmax	74,8 MHz (alvo 27 MHz; TNS = 0)
Potência	197,5 mW (122,8 estática + 74,7 dinâmica)

Na FPGA, a RAM usa BSRAM inicializada com o firmware. Um *script* faz tudo pela linha de comando: compilação do firmware, síntese, posicionamento e roteamento, *bitstream* e gravação (`programmer_cli`). As ligações são:

- **HC-SR04 [13]:** trabalha em 5 V. Ele é alimentado por duas pilhas (cerca de 7 V) através de um conversor DC-DC abaixador (*buck*) ajustado para 5 V. O sinal ECHO, de 5 V, chega ao pino da FPGA (3,3 V) por um divisor resistivo de dois resistores, $V = 5\,\text{V} \cdot R_2/(R_1 + R_2) \approx 3{,}3\,\text{V}$. O TRIG sai direto da FPGA, porque o sensor reconhece 3,3 V como nível alto.
- **VL53L0X:** no barramento I2C a 100 kHz, em 3,3 V.
- **GND:** comum a todos os componentes.
- **UART:** a 9.600 baud, a taxa do conversor USB da placa, identificada pela compressão de 12:1 do texto recebido a 115.200 baud.

A Fig. 14 mostra o circuito completo e o esquema da alimentação e do divisor.

A Tabela VI resume os recursos. O SoC usa 11% da lógica e 40% das BSRAM, ocupadas pela RAM de 16 KB e pelo banco de registradores. Ele fecha a 74,8 MHz, 2,8 vezes o clock usado. Na potência estimada (197,5 mW), a parcela estática é a maior (122,8 mW), típica da FPGA.

A Fig. 15 mostra a saída do SoC. Na placa:

- o autoteste dá 64/64 vetores iguais;
- todas as fusões com sensores reais dão hardware bit a bit igual ao software;
- os tempos medidos são 2.234 ciclos em software e 179 no coprocessador (12,5×).

A. Calibração dos sensores

Antes da calibração, com o obstáculo a 100 mm, os sensores divergiam em até cerca de 50 mm, muito acima dos 15 mm permitidos pela montagem (Fig. 16). O erro era quase todo *viés*:

- o laser lia de 24 a 38 mm a mais, o deslocamento que o fabricante recomenda calibrar [12];
- o ultrassom lia alguns milímetros a menos;
- os dois oscilavam pouco, com desvio padrão de 0,5 a 1,7 mm.

Como as eqs. (1)–(3) supõem média zero, o viés passaria direto para a estimativa.

```
reports/femto_soc_dft/femto_soc_scan_chains.rpt  (cabeçalho)
  Reporting 1 scan chain (muxed_scan)

  Chain 1: AutoChain_1
    scan_in:      dft_sdi_1
    scan_out:     dft_sdo_1
    shift_enable: SE (active high)
    clock_domain: clk (edge: rise)
    length: 2325

flip-flops do núcleo de fusão na cadeia (contados no mesmo arquivo)
  coprocessador (periph_fusion)      570 flip-flops  (grep -c 'u_fusion/')
    fusion_node                      358 flip-flops  (grep -c 'u_fusion/u_node/')
      fusion_gain                     74 flip-flops  (grep -c 'u_fusion/u_node/u_gain')
      fusion_channel                 104 flip-flops  (grep -c 'u_fusion/u_node/u_channel')
      fusion_variance                 50 flip-flops  (grep -c 'u_fusion/u_node/u_variance')
  CPU FemtoRV32                     1182 flip-flops  (grep -c 'u_cpu/')

exemplos de bits da cadeia no núcleo de fusão
      bit 1343     u_fusion/u_node/u_channel/innovation_1_reg[0]  <clk (rise)>
      bit 1447     u_fusion/u_node/u_gain/den_reg[0]  <clk (rise)>
      bit 1470     u_fusion/u_node/u_gain/k_reg[0]  <clk (rise)>
      bit 1521     u_fusion/u_node/u_variance/prod_1_reg[14]  <clk (rise)>

reports/femto_soc/femto_soc_area_hier.rpt  (SEM DFT)
    u_fusion       periph_fusion_W16_FRAC15                2861   9044.532  3682.693    12727.225
      u_node       fusion_node_W16_FRAC15                  2115   6718.248  2745.741     9463.989
        u_channel  fusion_channel_W16_FRAC15                800   2480.868  1064.424     3545.292
        u_gain     fusion_gain_W16_FRAC15                   311   1127.574   395.987     1523.561
        u_variance fusion_variance_W16_FRAC15               674   1911.438   769.168     2680.606

reports/femto_soc_dft/femto_soc_area_hier.rpt  (COM DFT)
    u_fusion       periph_fusion_W16_FRAC15                2923  12674.520  4136.458    16810.978
      u_node       fusion_node_W16_FRAC15                  2174   8965.188  3036.189    12001.377
        u_channel  fusion_channel_W16_FRAC15                812   3150.504  1143.204     4293.708
        u_gain     fusion_gain_W16_FRAC15                   343   1594.404   461.653     2056.057
        u_variance fusion_variance_W16_FRAC15               685   2242.836   849.031     3091.867
```

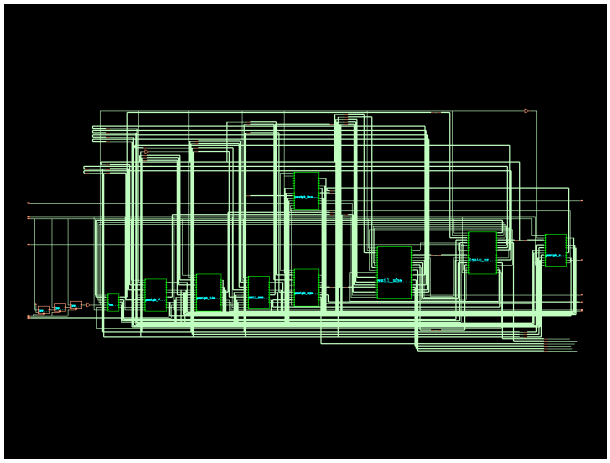


Figura 12. DFT no núcleo de fusão: trechos do relatório da cadeia (contagem por bloco e bits do ganho, do canal e da variância) e área do núcleo sem e com DFT (esq.); esquemático do `femto_soc` com DFT gerado pelo Genus (dir.).

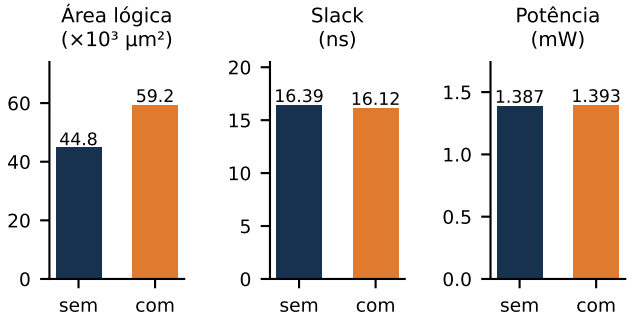


Figura 13. Impacto da DFT: área lógica (sem macros), folga e potência.

A calibração segue [1] e caracteriza cada sensor em distâncias conhecidas:

1) Um firmware dedicado acumula blocos de 80 amostras por sensor e reporta média, variância e falhas.
2) Em cada distância (seis distâncias, de 100 a 350 mm), foram colhidas mais de 10 mil medidas por sensor. Blocos com o obstáculo em movimento são descartados.
3) Uma ferramenta ajusta por mínimos quadrados o modelo bruta $= a \cdot$ real $+ b$ e gera os coeficientes de correção e a variância da medida corrigida, em ponto fixo, para o firmware.

As correções obtidas foram:

$$\text{US}: \quad \text{real} = 1{,}013\,\text{bruta} + 0{,}7\ \text{mm}, \tag{4}$$

$$\text{ToF}: \quad \text{real} = 1{,}007\,\text{bruta} - 27{,}9\ \text{mm}. \tag{5}$$

O erro residual, entre 100 e 350 mm, ficou em no máximo 5,5 mm no ultrassom e no máximo 7,0 mm no laser (Fig. 17).

Na placa, com o obstáculo a 200 mm (Fig. 15), as leituras corrigidas ficaram entre 196 e 201 mm (US) e entre 195 e 202 mm (ToF), e a divergência entre os sensores ficou em ± 5 mm. A estimativa fundida ficou entre 196 e 201 mm, com variância de 5 mm², menor que a de cada sensor ($K = 0{,}829$).

A variância do ultrassom usada nesse experimento (29 mm²) ficou inflada por blocos com o obstáculo em movimento a 150 mm. A versão final da ferramenta descarta esses blocos e soma à variância o erro residual da reta.

B. *Demonstração em vídeo*

Vídeo da demonstração (build, gravação, autoteste, leituras calibradas e fusão HW = SW com o obstáculo em movimento): https://youtu.be/17MWCjpcaUI?si=WISrPu6cReOIZggR

IX. Fluxo e Reprodutibilidade

Todo o fluxo é automatizado por *scripts*, e cada resultado deste artigo é reproduzido por um comando (Tabela VII).

O `synthesis.tcl` executa no Genus a sequência `read_hdl`, `elaborate`, `read_sdc`, `syn_generic`, `syn_map` e `syn_opt`. Com DFT, ele acrescenta `set_db dft_scan_style muxed_scan`, `define_shift_enable`, `check_dft_rules`, `connect_scan_chains` e `syn_opt -incremental`. Em seguida gera os relatórios de *timing*, potência, área, portas e regras, a *netlist* (`write_hdl`) e o SDF (`write_sdf`).

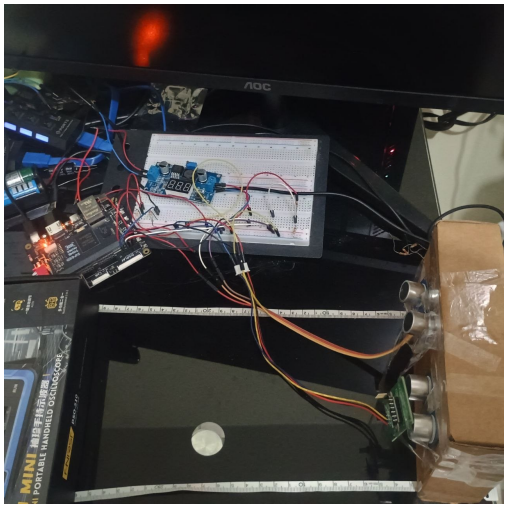
X. Resultados: Esperado versus Obtido

A Tabela VIII resume as etapas executadas e onde está a evidência de cada uma.

A Tabela IX confronta as metas com o que foi medido. Todas foram atingidas.

Na integração, a aceleração é limitada pelo acesso registrador a registrador pelo barramento. Com os oito nós e dados empacotados, a mesma transferência atende vários canais.

A divergência inicial entre os sensores não era falha da fusão, e sim viés dos sensores, eliminado pela calibração.



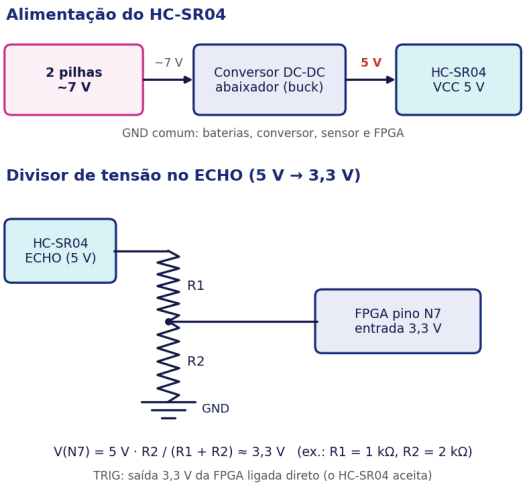


Figura 14. Circuito completo (esq.): Tang Primer 20K Dock, protoboard com o conversor DC-DC e o divisor de tensão, HC-SR04 e VL53L0X lado a lado, apontados para o obstáculo, com fita métrica. Esquema (dir.): duas pilhas (≈7 V) e conversor abaixador para os 5 V do HC-SR04; divisor de dois resistores no ECHO (5 V → 3,3 V).

```
==== femto_soc: FemtoRV32 + AXI4-Lite + fusao (27 MHz) ====
autoteste HW x SW: 64/64 iguais
VL53L0X ok
cal US: x1.01 ofs 1 var 29 | ToF: x1.00 ofs -28 var 6
#n US bruta>corr ToF bruta>corr dif | q var K | SW HW (n=nucleo) speedup
#  18 US 192> 196 ToF 227> 201 dif-  5 | q  200 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
#  35 US 192> 196 ToF 226> 200 dif-  4 | q  199 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
#  52 US 192> 196 ToF 228> 202 dif-  6 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
#  69 US 196> 200 ToF 226> 200 dif   0 | q  200 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
#  86 US 192> 196 ToF 223> 197 dif-  1 | q  197 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 103 US 196> 200 ToF 223> 197 dif   3 | q  198 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 120 US 197> 201 ToF 226> 200 dif   1 | q  200 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 137 US 196> 200 ToF 225> 199 dif   1 | q  199 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 154 US 192> 196 ToF 224> 198 dif-  2 | q  198 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 171 US 197> 201 ToF 225> 199 dif   2 | q  199 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 188 US 196> 200 ToF 226> 200 dif   0 | q  200 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 205 US 197> 201 ToF 227> 201 dif   0 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 222 US 196> 200 ToF 228> 202 dif-  2 | q  202 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 239 US 192> 196 ToF 223> 197 dif-  1 | q  197 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 256 US 192> 196 ToF 225> 199 dif-  3 | q  198 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 273 US 192> 196 ToF 229> 203 dif-  7 | q  202 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 290 US 192> 196 ToF 225> 199 dif-  3 | q  198 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 307 US 192> 196 ToF 226> 200 dif-  4 | q  199 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 324 US 192> 196 ToF 227> 201 dif-  5 | q  200 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 341 US 192> 196 ToF 226> 200 dif-  4 | q  199 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 358 US 197> 201 ToF 227> 201 dif   0 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 375 US 196> 200 ToF 227> 201 dif-  1 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 392 US 196> 200 ToF 231> 205 dif-  5 | q  204 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 409 US 197> 201 ToF 227> 201 dif   0 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 426 US 197> 201 ToF 223> 197 dif   4 | q  198 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 443 US 197> 201 ToF 228> 202 dif-  1 | q  202 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 460 US 197> 201 ToF 227> 201 dif   0 | q  201 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
# 477 US 196> 200 ToF 221> 195 dif   5 | q  196 var  5 K .829 | SW 2234c 82.7us HW 179c 6.6us n22 12.5x ok
```

Figura 15. Saída do SoC na UART (captura de tela) com o obstáculo a 200 mm: autoteste, leituras brutas e corrigidas, fusão, ciclos e aceleração.

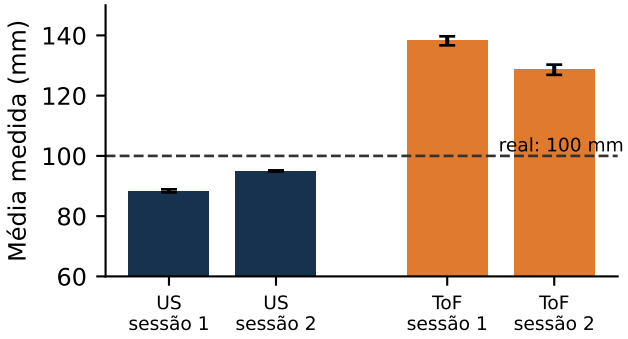


Figura 16. Média e desvio padrão a 100 mm antes da calibração: o erro é sistemático.

XI. Conclusão

Este trabalho percorreu o fluxo completo de um CI digital para a fusão sensorial: RTL, verificação funcional e UVM, síntese lógica com e sem DFT e protótipo calibrado em FPGA com sensores reais.

Os resultados principais são:

- **Desempenho:** o núcleo acelera a fusão em 102 vezes e, visto pelo software, em 12,5 vezes, com resultado idêntico ao software e 12,5 vezes menos energia por fusão.
- **Paralelismo:** com os oito nós simultâneos, a vazão chega a cerca de 20 vezes a da arquitetura de referência.
- **Testabilidade:** a DFT alcança 100% dos registradores varríveis com acréscimo de 5,9% de área.

O trabalho complementa Conde *et al.* [1] em três pontos:

Tabela VII
Scripts e comandos do projeto

Comando	Função
`./exe.sh pre MODULE=femto_soc`	simulação RTL (Xcelium)
`./exe.sh syn SYNTHESIS_WITH_DFT=yes`	síntese lógica (Genus)
`... syn MODULE=femto_sram`	síntese com cadeias de varredura
`./exe.sh post`	pós-síntese (*netlist* + SDF)
`./run_all_functional.sh`	regressão completa (8 cenários)
`frontend/firmware/Makefile`	firmware RV32I (`isa_test`, `c_suite`, SoC)
`fpga/.../build.sh all`	FPGA: firmware, síntese, P&R, *bitstream*, gravação
`tools/calib_session.sh`	coleta da calibração (distância real digitada)
`tools/calib_fit.py`	ajuste e geração do `calib.h`

- escala o núcleo de um para oito canais simultâneos, verificados por UVM e sintetizados com DFT;
- integra a fusão a um SoC RISC-V completo, com memória, barramento AXI4-Lite e sensores, funcionando em

Tabela VIII
ETAPAS EXECUTADAS, RESULTADO E EVIDÊNCIA NO PROJETO

Etapa	Ferramenta	Resultado	Evidência (arquivo)
Testes funcionais (RTL)	Xcelium	`isa_test`, `c_suite`, memórias, Quark com macros e SoC: 8/8 PASS	`backend/synthesis/work/pre/*.log` (Figs. 20, 21)
UVM	Xcelium + UVM	22/22 testes, 0 erros, cobertura funcional de 90,1% a 99,4%	`fusion_core/backend/uvm/*/reports/` (Figs. 23, 22)
Síntese lógica	Genus, GPDK045	SoC: 249.041 µm², folga de 16,4 ns, 1,39 mW; Quark, memórias e `fusion_core`	`backend/synthesis/reports/*/` (Figs. 8, 18)
DFT	Genus, *muxed scan*	SoC inteiro, inclusive o núcleo de fusão (570 FFs): 2.325 FFs em uma cadeia, 0 violações, 100% varríveis, +5,9% de área	`reports/femto_soc_dft/`, `deliverables/femto_soc_dft/atpg/` (Figs. 19, 12)
Pós-síntese	Xcelium + SDF	realizada na *netlist* da memória: 186/186 atrasos anotados, 0 erros, PASS. SoC sem e com DFT: fluxo pronto, execução pendente	`work/post/femto_sram/` (Fig. 6)
FPGA	Gowin EDA	11% da lógica, Fmax de 74,8 MHz, HW = SW na placa, 12,5×	`fpga/.../impl/`, `fpga/printscreens/` (Figs. 15, 19)

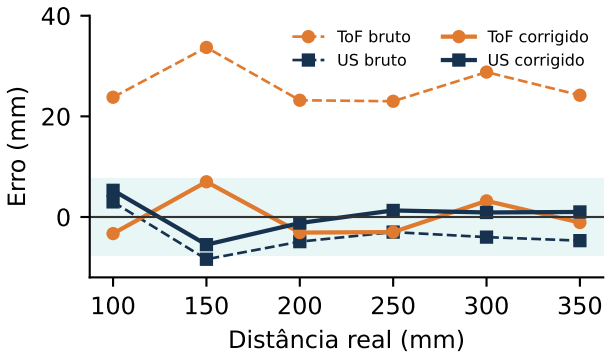


Figura 17. Erro de cada sensor antes (tracejado) e depois (contínuo) da calibração, de 100 a 350 mm. Faixa sombreada: ±7,5 mm.

Tabela IX
ESPERADO VERSUS OBTIDO

Item	Esperado	Obtido
Fusão em HW	igual ao modelo	bit a bit igual (UVM, sim., placa)
UVM	0 erros, cob. >90%	0 erros; 90,1–99,4%
Latência de um nó	dezenas de ciclos	22 ciclos (0,81 µs)
Aceleração	>10×	102× núcleo; 12,5× integrada
Vazão (8 nós)	> referência	72,7 M fusões/s (≈20×)
Timing ASIC	fechar a 27 MHz	folga de 16,4 ns
Área do nó	menor que a CPU	9.464 vs 21.928 µm²
DFT	0 violações	0; 100% varríveis; +5,9%
FPGA	SoC funcional	funcional; Fmax 74,8 MHz
Divergência US–ToF	≤15 mm	50 mm → ±5 mm calibrado

FPGA;

- adota o mesmo princípio de caracterização dos sensores, com uma calibração que separa viés e variância.

Os trabalhos futuros são:

- integrar os oito nós ao coprocessador do SoC na FPGA, com transferências empacotadas;
- migrar para AXI4 completo;
- executar o ATPG no Modus;
- fazer a simulação pós-síntese e a verificação de equivalência do SoC;
- portar o protótipo para Lattice.

APÊNDICE
EVIDÊNCIAS DOS RELATÓRIOS

As Figs. 18–22 reproduzem trechos dos relatórios gerados pelas ferramentas, como evidência dos números das Seções V a VIII. O texto foi copiado dos arquivos sem alteração; o caminho de cada arquivo aparece no título de cada bloco. Os esquemáticos do Genus estão nas Figs. 8 e 4, e a captura de tela da placa, na Fig. 15.

REFERÊNCIAS

[1] M. E. Conde, S. Cruz, D. M. Muñoz, C. H. Llanos e E. L. F. Fortaleza, "An efficient data fusion architecture for infrared and ultrasonic sensors, using FPGA," em *Proc. IEEE 4th Latin American Symposium on Circuits and Systems (LASCAS)*, 2013.

[2] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, n. 1, pp. 35–45, 1960.

[3] R. Siegwart e I. R. Nourbakhsh, *Introduction to Autonomous Mobile Robots*. Cambridge, MA: MIT Press, 2004.

[4] A. Waterman e K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, versão 20191213. RISC-V International, 2019.

[5] B. Levy e M. Koch, "FemtoRV32 Quark (`FemtoRV/RTL/PROCESSOR/femtorv32_quark.v`)," repositório *learn-fpga*, licença BSD-3-Clause, 2020–2021. [Online]. Disponível: https://github.com/BrunoLevy/learn-fpga

[6] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, IHI 0022.

[7] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu e M. Sarwar, "OpenRAM: an open-source memory compiler," em *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016.

[8] VLSI-DA (UC Santa Cruz e Oklahoma State University), "OpenRAM: an open-source memory compiler," versão 1.2.48, licença BSD-3-Clause. [Online]. Disponível: https://github.com/VLSIDA/OpenRAM

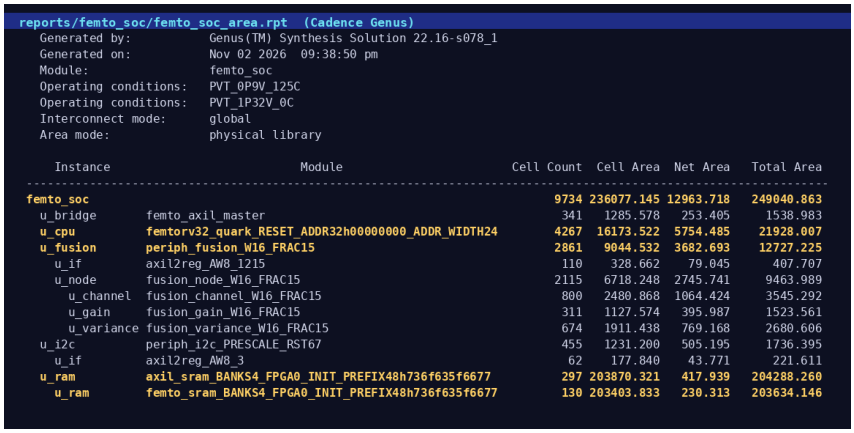
[9] J. E. Stine *et al.*, "FreePDK: an open-source variation-aware design kit," em *Proc. IEEE International Conference on Microelectronic Systems Education (MSE)*, 2007, pp. 173–174.

[10] Cadence Design Systems, *Generic Process Design Kit 45 nm (GPDK045) e biblioteca de células padrão gsclib045*, v4.7.

[11] IEEE, *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2, 2020.

[12] STMicroelectronics, *VL53L0X: World's smallest Time-of-Flight ranging and gesture detection sensor*, datasheet.

[13] ElecFreaks, *HC-SR04 Ultrasonic Ranging Module*, datasheet.



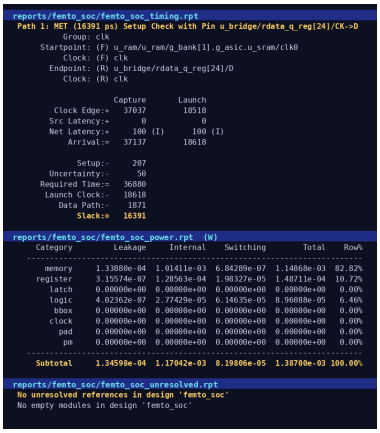
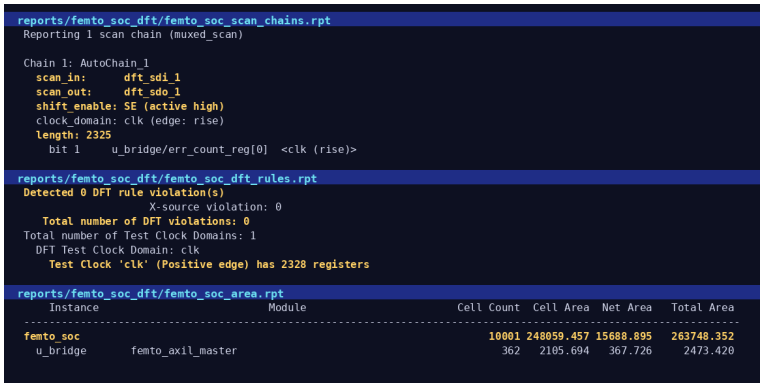


Figura 18. Genus, `femto_soc` sem DFT: área hierárquica (esq.); *timing*, potência e referências não resolvidas (dir.).



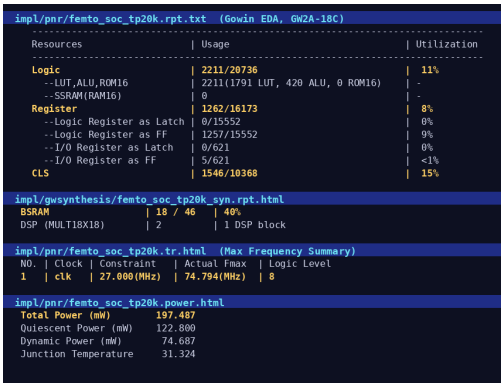
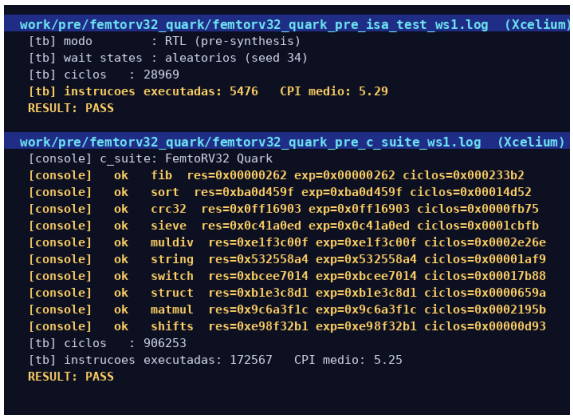


Figura 19. Genus com DFT: cadeia de varredura, regras e área (esq.). Gowin EDA: recursos, Fmax e potência na GW2A-18C (dir.).



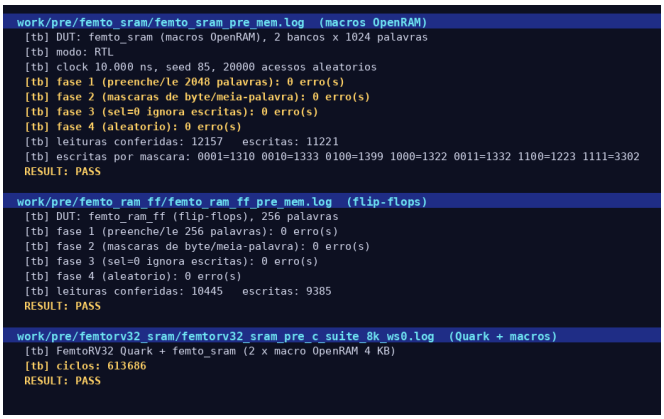
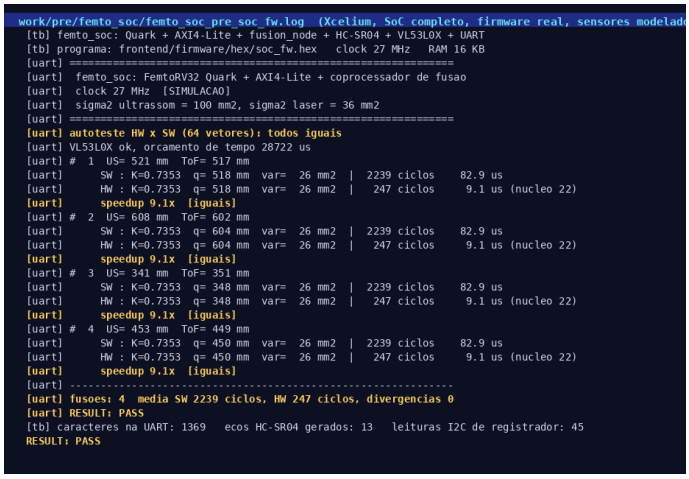


Figura 20. Xcelium, simulação RTL (trechos dos *logs*): processador com `isa_test` e `c_suite` com estados de espera (esq.); memórias em macros e em flip-flops e Quark executando das macros (dir.).

[14] Pololu, “VL53L0X library for Arduino,” [Online]. Disponível: https://github.com/pololu/vl53l0x-arduino
[15] Sipeed, “Tang Primer 20K (Gowin GW2A-LV18PG256C8/I7),” documentação da placa. [Online]. Disponível: https://wiki.sipeed.com



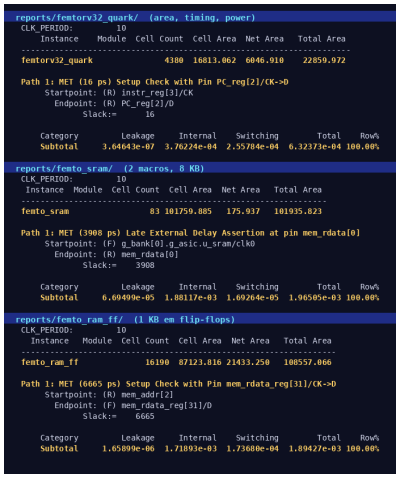
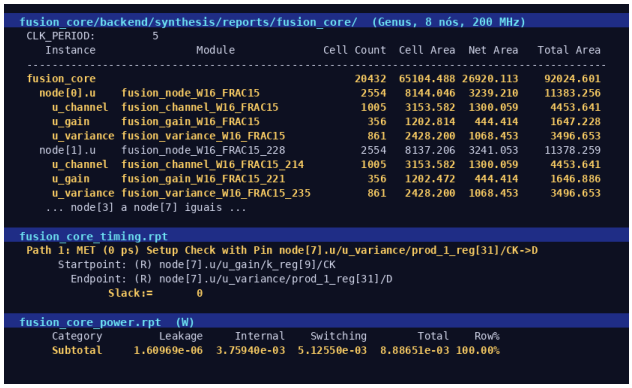


Figura 21. Xcelium, simulação do SoC completo com o firmware e os sensores modelados (esq.). Genus, síntese de cada parte a 100 MHz: processador e as duas memórias (dir.).



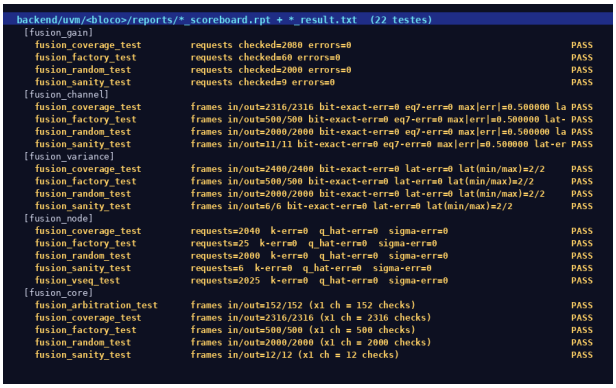


Figura 22. Genus, `fusion_core` com oito nós a 200 MHz: área por nó e por bloco, *timing* e potência (esq.). UVM: *scoreboard* e resultado dos 22 testes dos cinco ambientes (dir.).

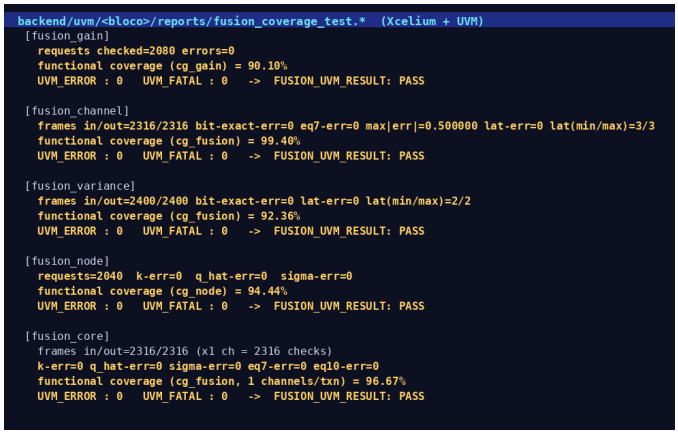


Figura 23. Xcelium + UVM: *scoreboard*, cobertura funcional e resultado do teste de cobertura de cada ambiente.